

학습한 ML/DL 모델 산출물

deal-paper-rf-ensemble-v1 · 2026-08-31

항목	내용
제품·역할	SalesLuv B2B 딜의 Won 확률 및 승패 예측
선정 후보	Stacking_LR
모델 버전	deal-paper-rf-ensemble-v1
모델 유형	RF·LR·ExtraTrees·CatBoost + LogisticRegression 메타모델
입력·임계값	13개 범주형 컬럼, 0.50
작성 기준일	2026-08-31
산출물 상태	1회차 Train 후보 저장·재로드 검증 완료, 서비스 미교체

1. 모델 카드

1.1 목적과 출력

구조화 Agent가 업무 보고서에서 추출한 13개 범주형 값을 받아 Won 확률을 계산하는 이진 분류 모델이다. 논문에서 선택한 RandomForest를 베이스라인 모델 계열로 삼았다. 학습 완료 파일과 세부 파라미터·튜닝 탐색 방법은 확인되지 않아, 공개된 8:2 분할·30회 반복 평가 방식을 참고해 RF를 직접 학습하고 우리 실험의 기준선으로 삼았다. 이후 우리가 추가한 GridSearch 튜닝과 양상블 비교로 발전시켰다. 최신 후보에는 TabICL이나 딥러닝 모델이 포함되지 않는다.

응답 항목	형식	연결 시 사용할 의미
won_probability	0~1 실수	모델의 P(Won)
predicted_class	Won / Lost	확률이 0.50 이상이면 Won
model_version	문자열	deal-paper-rf-ensemble-v1

위 표는 새 후보를 연결할 때의 출력 계약이며 현재 백엔드가 이 버전으로 교체되었다는 뜻은 아니다. 모델은 영업 판단을 지원하며 계약 승인·딜 상태 변경·담당자 평가를 자동으로 결정하지 않는다.

1.2 구성과 선정 이유

기본 모델	사용 이유	모델 내부 입력 처리
RandomForest	논문의 RF를 출발점으로 삼아 같은 조건에서 튜닝 효과 비교	고정 범주 원핫 인코딩
LogisticRegression	작은 데이터에서 규제 가능한 단순 선형 기준 제공	고정 범주 원핫 인코딩
ExtraTrees	무작위 분할 트리로 RF와 다른 예측 특성 제공	고정 범주 원핫 인코딩
CatBoost	범주형을 직접 처리하는 부스팅 계열 결합	원본 범주형 13개 직접 입력

네 모델의 `P(Won)`을 순서대로 모아 LogisticRegression 메타모델에 넣는다. 메타모델에는 원본 입력을 추가하지 않는다 (`passthrough=False`). 같은 원본 입력 그룹이 겹치지 않는 3-Fold OOF 확률로 결합 방법을 학습했다. 추론에는 저장된 `StackingClassifier.predict_proba()`를 사용하므로 서버에서 네 확률을 수동으로 조립할 필요가 없다.

OOF는 기본 모델의 학습 행과 메타모델 학습용 확률을 만드는 행을 나누는 절차다. 기본 모델 파라미터는 해당 바깥 Train 전체의 내부 CV로 선택했으므로, OOF 자체를 파라미터 탐색까지 완전히 분리한 평가 점수로 보고하지 않는다.

2. 입력 계약

2.1 필드 의미와 허용 범주

필드	의미	허용 범주
Authority	고객 의사결정권자의 권한 수준	High, Low, Mid, Unknown
Competitors	경쟁사 존재·검토 여부	No, Unknown, Yes
Purch_dept	고객사 구매부서 참여 여부	No, Unknown, Yes
Budgt_alloc	고객 예산 확보·배정 여부	No, Unknown, Yes
Forml_tend	공식 입찰 절차 여부	No, Unknown, Yes
RFP	제안요청서 진행 여부	No, Unknown, Yes
Posit_statm	고객의 명시적 구매 표현	Neutral, No, Unknown, Yes
Source	영업기회 유입 경로	Direct mail, Event, Joint past, Media, Online form, Other, Referral, Unknown
Client	고객과의 거래 관계	Current, New, Past, Unknown
Scope	계약·수행 범위의 명확성	Clear, Few questions, Low, Unknown
Cross_sale	기존 고객 대상 교차판매 여부	No, Unknown, Yes
Deal_type	거래 유형	Consulting, Maintenance, Project, Solution, Unknown
Needs_def	고객 요구사항의 정의 수준	Info gathering, No, Poor, Unknown, Yes

2.2 전처리·검증 규칙

1. 앞뒤 공백을 정리하고 측정되지 않은 값은 문자열 `Unknown`으로 통일한다. 미확인을 `No`로 바꾸지 않는다.
2. 저장된 `model_feature_names` 순서로 13개 필드를 구성한다. 허용 범주 밖의 문자열·오타자는 입력 오류로 처리한다.
3. RF·LR·ExtraTrees의 Pipeline은 고정 범주와 `drop="first"`로 39개 숫자 컬럼을 만든다. CatBoost에는 13개 문자열 컬럼을 그대로 전달한다.
4. `classes_`에서 1의 확률 열을 찾아 `P(Won) >= 0.50`으로 판정한다. 정확히 0.50일 때도 평가 코드와 같은 결과가 되게 한다.

실제 추론에서는 임의로 네 컬럼을 가리지 않는다. 네 개 Unknown 마스킹은 학습·평가 실험용이다. 운영에서는 구조화 Agent가 확인하지 못한 항목만 Unknown으로 전달한다. 모델 파일을 읽는 것만으로 모든 입력 검증이 자동 제공되지는 않으므로 새 백엔드 연결에서도 이 경계를 유지해야 한다.

3. 학습 범위와 저장된 파라미터

원본 448건과 13개 입력을 유지했다. 같은 원본 입력 198개 그룹을 Train 158그룹 / Test 40그룹으로 나누는 30회 반복 평가를 수행했다. 그룹을 약 8:2로 나누므로 실제 행 수는 회차마다 다르다. 모든 후보에 같은 분할과 같은 10개 마스킹 세트를 적용했다.

RF는 각 바깥 Train에서 64개 조합을 5-Fold 그룹 CV로 탐색했다. 추가 모델도 같은 바깥 Train 내부에서 LR 8개, ExtraTrees 18개, CatBoost 18개 조합을 탐색했다. 파라미터 선택 기준은 `neg_brier_score`다. 임계값과 Soft Voting의 동일 가중치는 별도 튜닝하지 않았다.

저장 모델은 성능이 가장 높은 회차가 아니라 **미리 정한 1회차 Train**으로 학습한 후보다. 원본 357건의 10개 마스킹 버전인 3,570행을 사용했고 나머지 91건은 학습하지 않았다. 전체 448건 재학습은 아직 수행하지 않았다.

저장 객체	1회차 Train에서 선택한 주요 파라미터
RandomForest	<code>n_estimators=300, max_depth=12, min_samples_leaf=5, max_features="sqrt"</code>
LogisticRegression	<code>C=0.1, class_weight=None, max_iter=3000, solver="lbfgs"</code>
ExtraTrees	<code>n_estimators=300, max_depth=12, min_samples_leaf=6, max_features="sqrt"</code>
CatBoost	<code>iterations=200, depth=4, learning_rate=0.05, l2_leaf_reg=3</code>
메타모델	<code>LogisticRegression(C=0.1, max_iter=3000, solver="lbfgs")</code>

각 바깥 Train마다 최적 파라미터를 다시 선택했다. 위 한 세트의 파라미터로 30회 전체 성능을 얻었다고 해석하면 안 된다. 재현 난수는 1이며 RF·ExtraTrees·CatBoost는 CPU로 실행했다. 현재 후보의 실행에는 MPS나 TabICL 체크포인트가 필요하지 않다.

4. 성능 근거와 선택 해석

논문 RF의 Accuracy는 78.2%, AUC는 0.85였다. 이 값을 우리가 재현한 결과로 쓰지 않고, 같은 13개 입력·마스킹·그룹 분리 조건에서 직접 학습한 RF 성적을 개선의 출발점으로 사용했다.

발전 단계	Accuracy ↑	AUC ↑	Brier ↓
자체 학습 RF 베이스라인	0.695847	0.740949	0.209706
RF 하이퍼파라미터 튜닝	0.733833	0.775560	0.190198
최종 선정 Stacking	0.735980	0.778318	0.187965

최종 후보는 자체 베이스라인보다 Accuracy **4.01%p**, AUC **0.037369**가 개선됐고 Brier는 **0.021741** 낮아졌다. 이는 우리 조건 안의 개선이며, 입력과 평가 조건이 다른 논문보다 우수하다는 뜻은 아니다.

다음 표는 위 최종 후보의 반복 평균과 실제 저장한 1회차 학습본을 구분한다.

지표	Stacking 30회 평가 평균	저장 모델의 1회차 Test 평균
Brier ↓	0.187965	0.206093
ROC-AUC ↑	0.778318	0.732313
Accuracy ↑	0.735980	0.716484
Precision ↑	0.711869	0.714017

지표	Stacking 30회 평가 평균	저장 모델의 1회차 Test 평균
Recall ↑	0.825865	0.789796
F1 ↑	0.762654	0.749823
FP / FN	16.15 / 8.09	15.5 / 10.3
FPR ↓	0.370101	0.369048

매 회차 Test의 10개 결측 패턴을 먼저 평균하고 그 값을 30회 평균했다. 10세트는 같은 거래를 공유하고 30회 분할도 서로 겹친다. 독립적인 새 거래 300세트나 독립된 최종 검증으로 해석하지 않는다. FP/FN은 평균 건수이므로 소수이며, 회차별 Test 규모 차이를 보완하려고 $FPR=FP/(FP+TN)$ 도 함께 본다.

Brier는 예측 확률의 제곱오차다. 실제 Lost인 거래를 높은 확률의 Won으로 확신할수록 불이익을 준다. 분류 문제라도 화면에 승률을 표시하므로 사용했다. 낮은 Brier만으로 확률 보정이 완벽하다고 보장할 수 없고, 모든 모델의 내부 학습 손실을 Brier로 바꾼 것도 아니다.

양상블은 튜닝 RF보다 평균 Brier·Accuracy·AUC·FP·FPR 중 어느 것도 나빠지지 않고 한 항목 이상 좋아진 후보 중 최소 Brier를 선택했다. Stacking은 튜닝 RF 대비 Accuracy 0.21%p, AUC 0.002758이 개선됐고, Brier 0.002233 및 평균 FP 0.62건이 감소했다. 대신 FN은 0.45건 증가했다. CatBoost가 Accuracy 1위, 4개 모델 Soft Voting이 AUC 1위이므로 Stacking이 모든 지표에서 최고라는 뜻은 아니다.

FP는 실제로 주의가 필요한 Lost 거래를 쉽게 성사될 거래로 잘못 표시하는 오류다. 위험 신호·개입 시점을 놓치고 영업 우선순위와 예상 매출을 낙관적으로 판단할 수 있어 중요하게 봤다. 다만 Stacking의 FP 16.15는 튜닝 RF 16.77보다 낮을 뿐, 튜닝 전 RF 15.59보다 낮지는 않다. 임계값을 높여 FP만 줄이는 실험은 이번 선정에 사용하지 않았다.

비교 결과를 보고 모델 계열을 선택했으므로 성능은 **탐색적 비교 결과**다. 작은 차이의 통계적 우위나 실제 고객 환경의 성능을 확정한 것은 아니다. 전체 비교와 논문 조건의 차이는 **학습결과서**에 기록했다.

5. 저장 파일과 로드 계약

현재 후보 파일은 하나다.

backend/pipeline/artifacts/deal-paper-rf-ensemble-v1.joblib

크기는 **17,139,030 bytes (16.345 MiB)**이며 SHA-256은 다음과 같다.

609c5d63b201fcb125cca9cddc2fcbe229f76d3ebf0a1417466d027248b17681

파일에는 학습한 **model**, 13개 입력 스키마, 허용 범주, 임계값, 1회차 Train/Test 위치, 선택 파라미터, 30회 평가 결과, 패키지 버전이 함께 들어 있다. 추론에는 번들의 **model**을 사용하며 별도 TabCL **.pkl**은 필요하지 않다.

RF 베이스라인·튜닝 파일은 실험 재현용이며 양상블 객체를 로드해 예측할 때 추가로 읽을 필요는 없다.

실험 재현 파일	역할
deal-paper-rf-baseline-v1.joblib	튜닝 전 RF와 동일 분할 기준
deal-paper-rf-tuned-v1.joblib	회차별 RF 최적 파라미터와 평가 결과

모델·데이터 파일은 Git에 포함하지 않는다. joblib은 로드 과정에서 코드를 실행할 수 있으므로 신뢰할 수 있는 파일만 사용하고 SHA-256을 먼저 확인한다. 다시 저장하면 해시가 달라질 수 있어 해당 버전의 기록을 함께 갱신해야 한다.

6. 확인한 사항과 남은 범위

확인한 사항:

- 최신 앙상블 노트북의 새 커널 전체 실행과 결과 저장
- 원본 입력 그룹의 바깥 Train/Test 및 내부 CV·OOF 비중복
- RF 튜닝 파일의 300개 Test 마스킹 결과 재현
- 7개 후보 × 30회 × 10세트, 총 2,100개 결과의 집계 일치
- 저장 객체의 1회차 Test 10세트 확률·지표 재현
- 별도 프로세스 재로드와 합성 정상·4개 Unknown·전체 Unknown 입력 추론
- 기존 RF 베이스라인·튜닝 산출물의 SHA-256 불변

전체 Unknown 입력에서 유효한 확률이 나오는 검사는 실행 가능성만 확인한다. 정보가 없는 입력에서도 예측 정확도가 검증됐다는 뜻은 아니다.

아직 수행하지 않은 범위: 전체 448건 재학습, 새 후보의 Linux/AWS 배포 검사, 실제 서비스 로더 교체, 운영 고객 데이터 평가. 구형 모델의 Linux·MPS 이식 검사를 이 후보의 검증 결과로 옮겨 적지 않았다.

7. 재현 환경과 실행 순서

재현 환경은 Python 3.13.13, scikit-learn 1.9.0, numpy 2.5.2, pandas 3.0.5, CatBoost 1.2.10, joblib 1.5.3이다.

노트북 전용 환경을 사용한다. 환경 파일은 `backend/notebooks/pyproject.toml`, `uv.lock`이며 실행 순서는 다음과 같다.

1. `deal_data_preprocessing.ipynb`: 13개 입력·마스킹 10세트·그룹 준비
2. `deal_model_paper_rf_baseline.ipynb`: RF 베이스라인, 30회 그룹 분리 평가
3. `deal_model_paper_rf_tuning.ipynb`: 각 Train 내부 RF 튜닝
4. `deal_model_paper_rf_ensemble.ipynb`: 추가 모델 튜닝·앙상블 비교·후보 저장

데이터 원본 SHA-256:

```
8dee635b95bdcdb00896b654efe62fc20177090081c81ef5224e8641ba31c3061
```

출처는 Salvirt B2B Sales Dataset이다. 원본 CSV는 로컬에서만 보관하며 상용 이용·재배포 조건은 별도로 확인한다. 전 처리의 구형 7:3 분할 출력은 이전 실험 호환용이고 최신 RF 실험에는 사용하지 않는다.

8. 백엔드 연결 시 할 일

현재 서비스 로더는 구형 `deal-stacking-lr-v1`을 읽는다. 새 후보는 번들 구조도 달라 파일명만 바꾸면 연결되는 상태가 아니다. 이번 문서 정리에서는 백엔드 실행 동작과 기존 배포 파일을 바꾸지 않았다.

- 최종 학습 범위를 정하고 전체 재학습 시 평가용 후보와 새 배포 버전을 구분한다.
- 새 번들의 모델·스키마·버전·임계값을 읽는 로더와 입력 검증을 연결한다.
- `predict_proba()`의 Won 결과 ≥ 0.50 판정이 평가 코드와 같은지 검사한다.
- 배포 환경에서 파일 해시·패키지 버전·로딩·추론을 확인한다.
- 실제 승패 확정 후 결측 패턴·Accuracy·AUC·Brier·FP/FN과 지연시간을 기록한다.

실제 고객 데이터의 수집·보관·재학습은 별도 개인정보 및 데이터 사용 범위를 확인한 뒤 진행한다.